

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №10
«Production Python: пакет, конфігурація, Docker, CI/CD»
з курсу «Професійний Python»
Варіант №1 — «Production-проект student-manager»**

Виконав: Лис Артур

Група: ФЕП-31

Перевірів(ла): _____

Дата: 08.09.2026

Львів — 2026

1. Мета роботи

Підготувати проєкт до production-використання: оформити пакет із wheel і sdist, винести конфігурацію та секрети в змінні середовища, додати /health і логування, контейнеризувати API в Docker, побудувати CI на GitHub Actions (lint, типи, тести, build, Docker) і задокументувати все в README.

2. Індивідуальний варіант і постановка задачі

Варіант 1 — student-manager: пакет student_manager, wheel/sdist, REST API студентів, конфігурація через DATABASE_URL, APP_ENV, LOG_LEVEL, Docker-образ, що запускає Student Manager API, /health, GitHub Actions з ruff, мурі, pytest, build і docker build; README з прикладами GET /students, POST /students, GET /groups/{id}/students.

Понад мінімум: multi-stage Docker build, non-root користувач, Docker HEALTHCHECK, docker-compose з volume, автоматичні міграції Alembic при старті контейнера, coverage із порогом у CI, матриця Python 3.12/3.13, артефакти wheel і coverage, семантична версія з CHANGELOG, захист мутацій секретним API_TOKEN, перевірка 401 у CI. Docker Desktop на робочій машині не встановлений, тому збірка образу та запуск контейнера виконуються і перевіряються в CI (job docker), а не локально — це чесно зафіксовано в розділі 8.

3. Проєкт і фінальна структура

Десять лабораторних зібрано в один пакет: analytics, streaming, domain, storage, notify, db, api, perf плюс settings.py цієї роботи. 234 тести, мурі strict, ruff.

```
student-manager/
├── .dockerignore
├── .env.example
├── .github/
│   └── workflows/
│       └── ci.yml
├── .gitignore
├── CHANGELOG.md
├── Dockerfile
├── README.md
├── alembic.ini
├── dist/
│   ├── student_manager-1.0.0-py3-none-any.whl
│   └── student_manager-1.0.0.tar.gz
├── docker-compose.yml
├── migrations/
│   ├── env.py
│   ├── script.py.mako
│   └── versions/
│       ├── 0001_initial_schema.py
│       └── 0002_add_student_email.py
├── pyproject.toml
├── src/
│   └── student_manager/
│       ├── __init__.py
│       └── analytics/
```

```

|      |— api/
|      |— cli.py
|      |— clock.py
|      |— config.py
|      |— db/
|      |— domain/
|      |— exceptions.py
|      |— main.py
|      |— models.py
|      |— notify/
|      |— perf/
|      |— registry.py
|      |— settings.py
|      |— snapshot.py
|      |— storage/
|      |— streaming/
|      |
|      |— tests/

```

4. pyproject.toml і метадані пакета

```

[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "student-manager"
version = "1.0.0"
description = "Облік студентів – наскрізний навчальний проєкт курсу «Професійний Python»"
readme = "README.md"
requires-python = ">=3.12"
authors = [{ name = "Artur Lys", email = "Artur.Lys@lnu.edu.ua" }]
license = "MIT"
keywords = ["students", "fastapi", "sqlalchemy", "education"]
classifiers = [
    "Development Status :: 5 - Production/Stable",
    "Intended Audience :: Education",
    "Programming Language :: Python :: 3",
    "Programming Language :: Python :: 3.12",
    "Programming Language :: Python :: 3.13",
    "Framework :: FastAPI",
    "Operating System :: OS Independent",
    "Typing :: Typed",
]
dependencies = ["pyyaml>=6.0"]

[project.optional-dependencies]
test = ["pytest>=8.0", "pytest-cov>=5.0", "pytest-asyncio>=0.24", "httpx>=0.27"]
db = ["sqlalchemy>=2.0", "alembic>=1.13"]
api = ["student-manager[db]", "fastapi>=0.115", "uvicorn>=0.30", "httpx>=0.27", "pydantic>=2.0"]
perf = ["numpy>=2.0"]
dev = ["mypy>=1.10", "types-PyYAML", "ruff>=0.6", "build>=1.2", "twine>=5.0"]

[project.scripts]
student-manager = "student_manager.main:main"
student-storage = "student_manager.storage.__main__:main"
student-api = "student_manager.api.__main__:main"

```

```
[project.urls]
Repository = "https://github.com/ArturLys/student-manager"
Changelog = "https://github.com/ArturLys/student-manager/blob/main/CHANGELOG.md"
```

Версія 1.0.0 за semantic versioning (MAJOR.MINOR.PATCH), історія у CHANGELOG.md; необов'язкові залежності розбито на групи db, api, perf, test, dev, щоб бібліотечний користувач не тягнув FastAPI. Три консольні команди; [tool.ruff] і [tool.pyru] тримають налаштування якості в тому ж файлі.

4.1. Збірка wheel і sdist, перевірка, встановлення у чисте середовище

```
$ python -m build
Successfully built student_manager-1.0.0.tar.gz and student_manager-1.0.0-py3-none-any.whl
```

```
$ ls -la dist
-rw-r--r-- 1 user 197610 131217 Sep  8 21:20 student_manager-1.0.0-py3-none-any.whl
-rw-r--r-- 1 user 197610 101962 Sep  8 21:20 student_manager-1.0.0.tar.gz
```

```
$ twine check dist/*
Checking dist/student_manager-1.0.0-py3-none-any.whl: PASSED
Checking dist/student_manager-1.0.0.tar.gz: PASSED
```

```
$ python -m venv /tmp/smoke && /tmp/smoke/Scripts/pip install "student-manager[api] @
file:///.../dist/student_manager-1.0.0-py3-none-any.whl"
import ok, version 1.0.0
$ student-api --help
usage: python -m student_manager.api [-h] {serve,demo} ...
```

```
positional arguments:
  {serve,demo}
    serve      запустити Uvicorn
    demo       прогнати сценарій через httpx.ASGITransport без сервера
```

```
options:
  -h, --help      show this help message and exit
$ pip show student-manager
Name: student-manager
Version: 1.0.0
Summary: Облік студентів – наскрізний навчальний проєкт курсу «Професійний Python»
Home-page:
```

5. Конфігурація через середовище та секрети

```
# Copy to .env and fill in. .env is ignored by Git; never commit real values.
# Database: SQLAlchemy URL (SQLite file by default; in Docker the volume is /data)
DATABASE_URL=sqlite:///data/students.db
# development | test | production (production refuses to start without API_TOKEN)
APP_ENV=development
# DEBUG | INFO | WARNING | ERROR | CRITICAL
LOG_LEVEL=INFO
# Bind address of the API server
API_HOST=127.0.0.1
API_PORT=8000
# Secret. When set, POST/PATCH/DELETE require the header X-API-Token: <value>
# Generate one: python -c "import secrets; print(secrets.token_urlsafe(32))"
API_TOKEN=
```

```

@dataclass(frozen=True, slots=True)
class Settings:
    database_url: str = DEFAULT_DATABASE_URL
    app_env: AppEnv = "development"
    log_level: str = "INFO"
    api_host: str = DEFAULT_API_HOST
    api_port: int = 8000
    api_token: str | None = None # secret: when set, POST/PATCH/DELETE require the X-API-Token header

    @classmethod
    def from_env(cls, environ: Mapping[str, str] | None = None) -> Settings:
        env = os.environ if environ is None else environ
        app_env = env.get("APP_ENV", "development").strip().lower()
        if app_env not in APP_ENVS:
            raise SettingsError(f"APP_ENV має бути одним із {' ', ' '.join(APP_ENVS)}, отримано
{app_env!r}")
        level = env.get("LOG_LEVEL", "INFO").strip().upper()
        if level not in LOG_LEVELS:
            raise SettingsError(f"LOG_LEVEL має бути одним із {' ', ' '.join(LOG_LEVELS)}, отримано
{level!r}")
        try:
            port = int(env.get("API_PORT", "8000"))
        except ValueError as error:
            raise SettingsError(f"API_PORT має бути цілим числом: {error}") from error
        token = env.get("API_TOKEN", "").strip() or None
        if app_env == "production" and token is None:
            raise SettingsError("у production API_TOKEN обов'язковий (передайте його через середовище,
не через код)")
        return cls(
            database_url=env.get("DATABASE_URL", "").strip() or DEFAULT_DATABASE_URL,
            app_env=app_env,
            log_level=level,
            api_host=env.get("API_HOST", DEFAULT_API_HOST),
            api_port=port,
            api_token=token,
        )

    def redacted(self) -> dict[str, object]:
        """Safe to log: secrets are masked, presence is still visible."""
        data = asdict(self)
        for key in SECRET_KEYS:
            if data.get(key):
                data[key] = "***"
        return data

```

Усі параметри читаються один раз при старті; невалідне значення або відсутній у production API_TOKEN зупиняє процес одразу (SettingsError), а не десь під навантаженням. Секрет ніколи не потрапляє в код: у репозиторії лише .env.example з порожнім значенням, .env — у .gitignore і .dockerignore, у логах redacted() показує ***. Порівняння токена — secrets.compare_digest, стійке до атак за часом.

```

def require_token(request: Request, x_api_token: Annotated[str | None, Header()] = None) -> None:
    """Mutating endpoints are open in development; with API_TOKEN set they need the matching header.
    compare_digest keeps the comparison constant-time, so the token cannot be guessed byte by byte."""
    settings: Settings = request.app.state.settings
    if settings.api_token is None:
        return

```

```

        if x_api_token is None or not secrets.compare_digest(x_api_token, settings.api_token):
            raise HTTPException(status.HTTP_401_UNAUTHORIZED, detail="потрібен коректний заголовок X-API-Token")

APP_ENV=production API_TOKEN=demo-secret LOG_LEVEL=WARNING -> Settings.redacted():
{"database_url": "sqlite:///data/students.db", "app_env": "production", "log_level": "WARNING",
"api_host": "127.0.0.1", "api_port": 8000, "api_token": "****"}
GET /health -> 200 {'status': 'ok', 'version': '1.0.0', 'app_env': 'production', 'database': 'ok'}
POST /students без токена -> 401 {'detail': 'потрібен коректний заголовок X-API-Token'}
POST /students з X-API-Token -> 201
GET /debug/slow y production -> 404

```

6. Логування та /health

```

def configure_logging(level: str) -> None:
    """One root configuration for the process; `force` replaces handlers left by libraries or earlier runs."""
    logging.basicConfig(
        level=level,
        format="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S",
        force=True,
    )

def register_health(app: FastAPI) -> None:
    @app.get("/health", tags=["ops"], response_model=HealthOut, summary="Liveness/readiness: процес і база даних")
    async def health(request: Request) -> HealthOut:
        settings: Settings = request.app.state.settings
        database_ok = await asyncio.to_thread(request.app.state.store.ping)
        return HealthOut(
            status="ok" if database_ok else "degraded",
            version=__version__,
            app_env=settings.app_env,
            database="ok" if database_ok else "error",
        )

```

/health відповідає на два питання оркестратора: чи живий процес і чи доступна база (SELECT 1 через store). Його викликають Docker HEALTHCHECK, docker-compose і CI.

7. Docker

```

# syntax=docker/dockerfile:1
# Stage 1: build the wheel from the sources (build tools never reach the runtime image)
FROM python:3.12-slim AS builder
WORKDIR /build
COPY pyproject.toml README.md ./
COPY src ./src
RUN pip install --no-cache-dir build && python -m build --wheel --outdir /build/dist

# Stage 2: slim runtime with a non-root user, the wheel and the migrations only
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    APP_ENV=production \
    LOG_LEVEL=INFO \
    DATABASE_URL=sqlite:///data/students.db \

```

```

    API_HOST=0.0.0.0 \
    API_PORT=8000
RUN useradd --create-home --uid 1000 app && mkdir -p /data && chown app:app /data
WORKDIR /app
COPY --from=builder /build/dist/*.whl /tmp/
COPY alembic.ini ./
COPY migrations ./migrations
RUN pip install --no-cache-dir "student-manager[api] @ file://$(ls /tmp/*.whl)" && rm -f /tmp/*.whl
USER app
VOLUME ["/data"]
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
    CMD python -c "import sys, urllib.request; sys.exit(0 if
urllib.request.urlopen('http://127.0.0.1:8000/health', timeout=2).status == 200 else 1)"
# migrations run at start-up, then the API (API_TOKEN must be passed with -e / an env file)
CMD ["sh", "-c", "alembic upgrade head && student-api serve"]

# .dockerignore
.git
.github
.venv
.env
.env.*
!.env.example
__pycache__
*.pyc
.pytest_cache
.mypy_cache
.ruff_cache
.coverage
htmlcov
dist
build
*.egg-info
data
logs
tests
Dockerfile
docker-compose.yml

services:
  api:
    build: .
    image: student-manager:1.0.0
    ports:
      - "8000:8000"
    env_file:
      - .env          # DATABASE_URL, APP_ENV, LOG_LEVEL, API_TOKEN — see .env.example
    volumes:
      - students-data:/data
    healthcheck:
      test: ["CMD", "python", "-c", "import sys, urllib.request; sys.exit(0 if
urllib.request.urlopen('http://127.0.0.1:8000/health', timeout=2).status == 200 else 1)"]
      interval: 30s
      timeout: 3s
      retries: 3
    restart: unless-stopped

```

```
volumes:
  students-data:
```

Два етапи: у builder збирається wheel, у runtime-образ потрапляють лише wheel, alembic.ini і migrations/; процес працює від користувача app, база лежить у volume /data, CMD спочатку виконує alembic upgrade head, потім запускає API. Конфігурація передається -e/env_file, а не вшивається в образ.

```
$ docker build -t student-manager:1.0.0 .
$ docker run -d -p 8000:8000 -e API_TOKEN=change-me -e LOG_LEVEL=DEBUG -v students-data:/data student-manager:1.0.0
$ curl http://localhost:8000/health
{"status":"ok","version":"1.0.0","app_env":"production","database":"ok"}
```

8. GitHub Actions

```
name: CI
```

```
on:
```

```
  push:
    branches: ["**"]
    tags: ["v*"]
  pull_request:
```

```
permissions:
  contents: read
```

```
jobs:
```

```
  lint:
    name: ruff (lint + format check)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install ruff
      - run: ruff check .
      - run: ruff format --check .
```

```
  build:
```

```
    name: wheel + sdist
    needs: [lint, typecheck, test]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install build twine
      - run: python -m build
      - run: twine check dist/*
      - name: install the wheel into a clean environment and smoke-test the entry points
        run: |
          python -m venv /tmp/smoke
          /tmp/smoke/bin/pip install --quiet "student-manager[api] @ file://$(pwd)/$(ls dist/*.whl)"
```

```

/tmp/smoke/bin/student-api --help
/tmp/smoke/bin/python -c "import student_manager; print('student_manager',
student_manager.__version__)"
- uses: actions/upload-artifact@v4
  with:
    name: dist
    path: dist/

docker:
  name: docker build + /health
  needs: [build]
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: docker build -t student-manager:${{ github.sha }} -t student-manager:latest .
    - name: run the container with configuration from the environment
      run: |
        docker run -d --name api -p 8000:8000 \
          -e API_TOKEN=${{ secrets.API_TOKEN || 'ci-only-token' }} -e LOG_LEVEL=DEBUG student-
manager:latest
        for i in $(seq 1 20); do
          if curl -fsS http://localhost:8000/health; then echo; break; fi
          sleep 1
        done
        curl -fsS http://localhost:8000/health | tee health.json
        grep -q '"status":"ok"' health.json
        echo "--- unauthorised POST must be 401 ---"
        test "$(curl -s -o /dev/null -w '%{http_code}' -X POST http://localhost:8000/students -H
'Content-Type: application/json' -d '{}')" = "401"
        docker image ls student-manager
        docker ps --filter name=api
    - if: always()
      run: docker logs api || true
    - if: always()
      run: docker rm -f api || true

```

П'ять job-ів: lint (ruff check + format check), typecheck (mypy strict), test (матриця 3.12/3.13, branch coverage з порогом 80 % із pyproject, артефакт coverage.xml), build (wheel + sdist, twine check, встановлення wheel у чистий venv і запуск student-api --help, артефакт dist/), docker (збірка образу, запуск контейнера з API_TOKEN із GitHub Secret, curl /health, перевірка 401 без токена, логи).

8.1. Результати pipeline

```

$ gh run view 34262648481
✓ main CI · 34262648481
Triggered via push about 3 minutes ago
JOBS
✓ pytest (Python 3.12) in 41s (ID 102184185898)
✓ mypy (strict) in 34s (ID 102184186061)
✓ pytest (Python 3.13) in 28s (ID 102184186144)
✓ ruff (lint + format check) in 8s (ID 102184186145)
✓ wheel + sdist in 28s (ID 102184428174)
✓ docker build + /health in 22s (ID 102184601798)
ANNOTATIONS
pytest (Python 3.12): .github#2
mypy (strict): .github#2

```

```

pytest (Python 3.13): .github#2
ruff (lint + format check): .github#2
wheel + sdist: .github#2
docker build + /health: .github#2
ARTIFACTS
dist
coverage-xml
For more information about a job, try: gh run view --job=<job-id>
View this run on GitHub: https://github.com/ArturLys/student-manager/actions/runs/3426264841

```

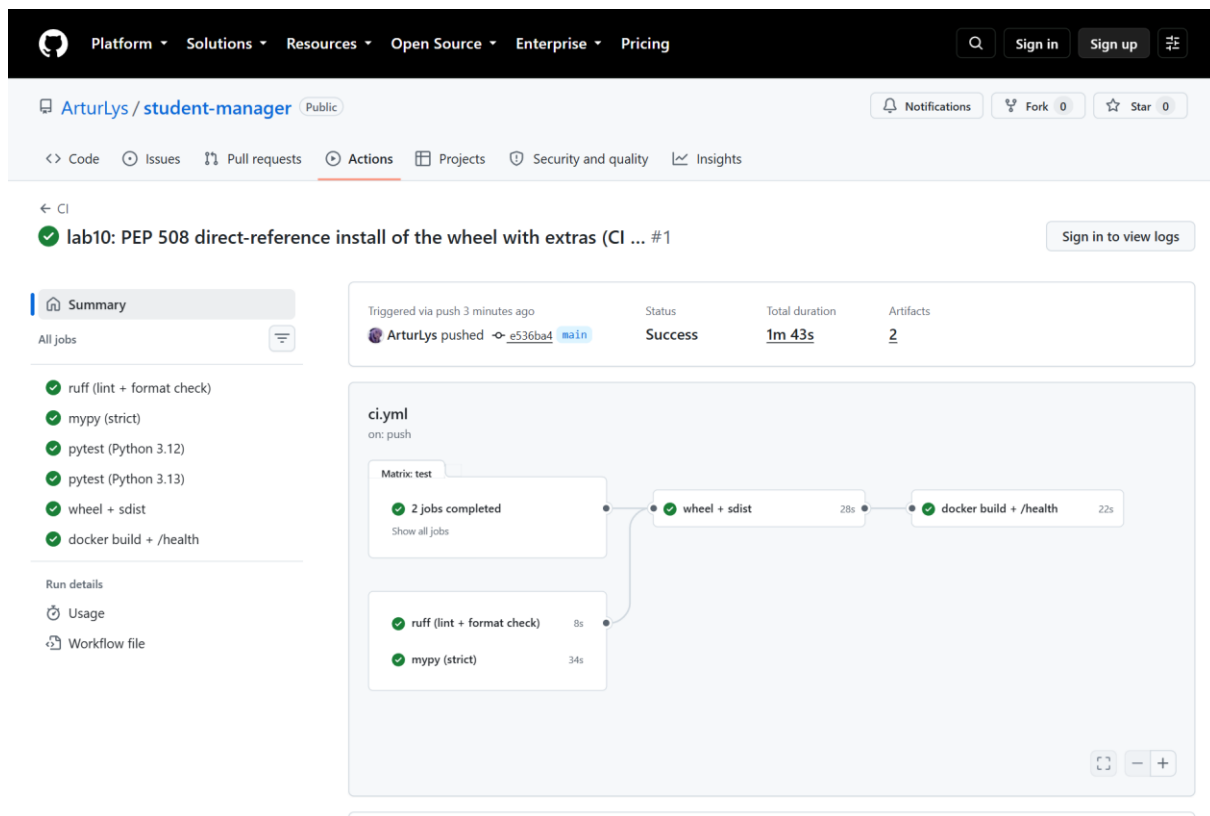


Рис. 1. Зелений pipeline на GitHub Actions (ArturLys/student-manager)

```

$ gh run view --job 102184601798 --log # job "docker build + /health", фрагмент
#14 1.641 Successfully installed build-1.6.0 packaging-26.3 pyproject_hooks-1.2.0
#14 3.473 Successfully built student_manager-1.0.0-py3-none-any.whl
#19 5.595 Successfully installed Mako-1.4.1 MarkupSafe-3.0.3 alembic-1.19.2 annotated-doc-0.0.5
annotated-types-0.8.0 anyio-4.15.1 certifi-2
#20 naming to docker.io/library/student-manager:e536ba41fed63db2857eb27a2560fd15018f8124 done
#20 naming to docker.io/library/student-manager:latest done
^[[36;1mecho "--- unauthorised POST must be 401 ---"^[0m
{"status":"ok","version":"1.0.0","app_env":"production","database":"ok"}
{"status":"ok","version":"1.0.0","app_env":"production","database":"ok"}--- unauthorised POST must be
401 ---
student-manager e536ba41fed63db2857eb27a2560fd15018f8124 fedf4cf249d4 3 seconds ago 176MB
student-manager latest fedf4cf249d4 3 seconds ago 176MB
bf2fa6202ddf student-manager:latest "sh -c 'alembic upgr..." 2 seconds ago Up 2 seconds (health:
starting) 0.0.0.0:8000->8000/tcp,
INFO: Application startup complete.

```

9. Локальні перевірки якості

```
$ ruff check . && ruff format --check .
```

```
warning: The following rules have been removed and ignoring them has no effect:  
- UP038
```

```
All checks passed!
```

```
warning: The following rules have been removed and ignoring them has no effect:  
- UP038
```

```
106 files already formatted
```

```
$ mypy src
```

```
Success: no issues found in 85 source files
```

```
$ pytest -q
```

```
..... [100%]  
234 passed in 19.06s
```

```
$ pytest tests/integration/test_production.py -v
```

```
tests/integration/test_production.py::test_settings_defaults_come_from_an_empty_environment PASSED  
tests/integration/test_production.py::test_settings_read_every_variable_and_redact_the_secret PASSED  
tests/integration/test_production.py::test_invalid_settings_fail_fast[bad-env] PASSED  
tests/integration/test_production.py::test_invalid_settings_fail_fast[bad-level] PASSED  
tests/integration/test_production.py::test_invalid_settings_fail_fast[bad-port] PASSED  
tests/integration/test_production.py::test_invalid_settings_fail_fast[production-without-token] PASSED  
tests/integration/test_production.py::test_dotenv_does_not_override_real_environment PASSED  
tests/integration/test_production.py::test_health_reports_version_environment_and_database PASSED  
tests/integration/test_production.py::test_mutations_require_the_token_when_it_is_configured PASSED  
===== 9 passed in 1.80s =====
```

10. README і Git

```
### Конфігурація через середовище (`.env.example`)
```

Змінна	Типово	Призначення
`DATABASE_URL`	`sqlite:///data/students.db`	SQLAlchemy URL бази
`APP_ENV`	`development` / `test` / `production`	(у production `/debug` вимкнено, `API_TOKEN` обов'язковий)
`LOG_LEVEL`	`INFO`	рівень логування
`API_HOST`, `API_PORT`	`127.0.0.1`, `8000`	адреса сервера
`API_TOKEN`	— **секрет**; коли задано, POST/PATCH/DELETE потребують заголовка `X-API-Token`	

Секрети живуть лише в середовищі або у локальному `.env` (у `.gitignore`); у код і в Git потрапляє тільки `.env.example`.

```
### Приклади API
```

```
```bash  
curl http://localhost:8000/students?group=ФЕП-31c&sort=-grade
curl -X POST http://localhost:8000/students -H "Content-Type: application/json" -H "X-API-Token: change-me" \
-d '{"first_name":"Марта","last_name":"Гнатишин","group":"ФЕП-31c","average_grade":93.4}'
```

```
curl http://localhost:8000/groups/0EP-31c/students
```

```
...
```

```
$ git log --oneline -6
```

```
9935ad5 lab10: README – production section (package, env, Docker, API examples, CI, checklist)
```

```
e536ba4 lab10: PEP 508 direct-reference install of the wheel with extras (CI smoke test, Dockerfile)
```

```
8320bdd lab10: fix settings defaults with slots, add store.ping for /health, static version, find-links
install in CI and Dockerfile
```

```
1ec1543 lab10: Dockerfile (multi-stage, non-root, healthcheck), .dockerignore, docker-compose, GitHub
Actions CI, CHANGELOG
```

```
964682a lab10: settings from environment, .env.example, /health, API token guard, semantic version
1.0.0, ruff config + format
```

```
78bae4d lab9: correctness tests for every implementation, README section, version 0.9.0, тупу
python_version 3.12
```

## 11. Production checklist

- Пакет: src-layout, метадані, семантична версія 1.0.0, CHANGELOG, wheel і sdist проходять twine check, встановлення у чисте середовище перевірено локально і в CI.
- Конфігурація: лише через середовище (DATABASE\_URL, APP\_ENV, LOG\_LEVEL, API\_HOST, API\_PORT, API\_TOKEN), валідація при старті, .env.example у репозиторії, .env — ні.
- Секрети: у коді відсутні, у логах масковані, у CI — GitHub Secret, у Docker — -e/env\_file.
- Спостережуваність: /health із перевіркою бази, структуроване логування з рівнем із середовища.
- Docker: multi-stage, non-root, HEALTHCHECK, volume для даних, міграції при старті, .dockerignore.
- CI: lint + format, типи, тести на 3.12 і 3.13 з покриттям  $\geq 80\%$ , build, Docker build і smoke-тест контейнера.
- Документація: README з прикладами endpoint-ів, Swagger /docs і /openapi.json генеруються FastAPI.

## 12. Відповіді на контрольні запитання

### 1. Чим wheel відрізняється від sdist?

Wheel (.whl) — готовий до встановлення бінарний дистрибутив; sdist (.tar.gz) — вихідний код, який збирається під час встановлення.

### 2. Що містять [build-system] і [project]?

Бекенд і його залежності для збірки (setuptools); метадані пакета: ім'я, версія, залежності, скрипти, класифікатори.

### 3. Що таке semantic versioning?

MAJOR — несумісні зміни, MINOR — нова сумісна функціональність, PATCH — виправлення; тут 1.0.0.

#### **4. Навіщо конфігурацію виносити з коду і що таке secret?**

Один образ працює в dev/test/prod без перезбірки; секрет — дані доступу (токени, паролі, ключі), витік яких дає доступ чужим.

#### **5. Чому .env не можна комітити і навіщо .env.example?**

У Git історія вічна — секрет лишиться назавжди; example показує, які змінні потрібні, без значень.

#### **6. Чим image відрізняється від container?**

Образ — незмінний шаблон із файловою системою; контейнер — запущений процес з цього образу.

#### **7. Що роблять WORKDIR, COPY, EXPOSE, CMD?**

Робочий каталог; копіювання файлів у образ; документування порту; команда за замовчуванням при старті.

#### **8. Для чого .dockerignore і volume?**

Не тягнути в контекст збірки .git, .venv, тести, секрети; volume зберігає дані (SQLite) поза контейнером.

#### **9. Що таке multi-stage build і non-root user?**

Збірка в одному образі, запуск в іншому, меншому і без build-інструментів; процес без root обмежує шкоду від уразливості.

#### **10. Що таке CI, CD, workflow, job, step, runner?**

Автоматична перевірка кожної зміни; автоматичне розгортання; файл у .github/workflows; паралельна одиниця на runner-і; окрема команда; машина GitHub, що виконує job.

#### **11. Для чого ruff і муру у CI?**

Ruff ловить помилки стилю та невикористаний код і тримає єдиний формат; муру перевіряє типи до запуску.

#### **12. Навіщо package build і Docker build у CI?**

Щоб зламаний pyproject або Dockerfile виявились на pull request, а не при релізі.

#### **13. Як використовувати GitHub Secret у workflow?**

`{{ secrets.NAME }}` у кроці; значення зберігається зашифрованим і маскується в логах.

#### 14. Чому logging краще за print?

Рівні, формат із часом і модулем, керування через LOG\_LEVEL, вимкнення без правки коду, робота з обробниками.

#### 15. Для чого health endpoint та Docker healthcheck?

Оркестратор бачить, чи процес живий і чи доступна база, і перезапускає нездоровий контейнер.

#### 16. Чим development відрізняється від production?

У production немає /debug, обов'язковий токен, стриманий рівень логів, конфігурація зі середовища, non-root контейнер.

#### 17. Що означає green CI pipeline?

Усі job-и завершилися успішно: код відформатовано, типи і тести проходять, пакет і образ збираються.

### 13. Висновок

Проект десяти лабораторних доведено до production-стану версії 1.0.0: пакет збирається у wheel і sdist та встановлюється в чисте середовище, конфігурація і секрети живуть у середовищі з валідацією при старті, API має /health і захист мутацій токеном, Docker-образ multi-stage з non-root користувачем і healthcheck запускає міграції та сервер, а GitHub Actions на кожен push проганяє ruff, муру, 234 тести на двох версіях Python із покриттям понад 80 %, збірку пакета і Docker-образ із перевіркою /health. Єдине обмеження — Docker перевірено лише в CI, бо локально Docker Desktop відсутній.